



Disciplina: **FGA314 – Testes de Software**

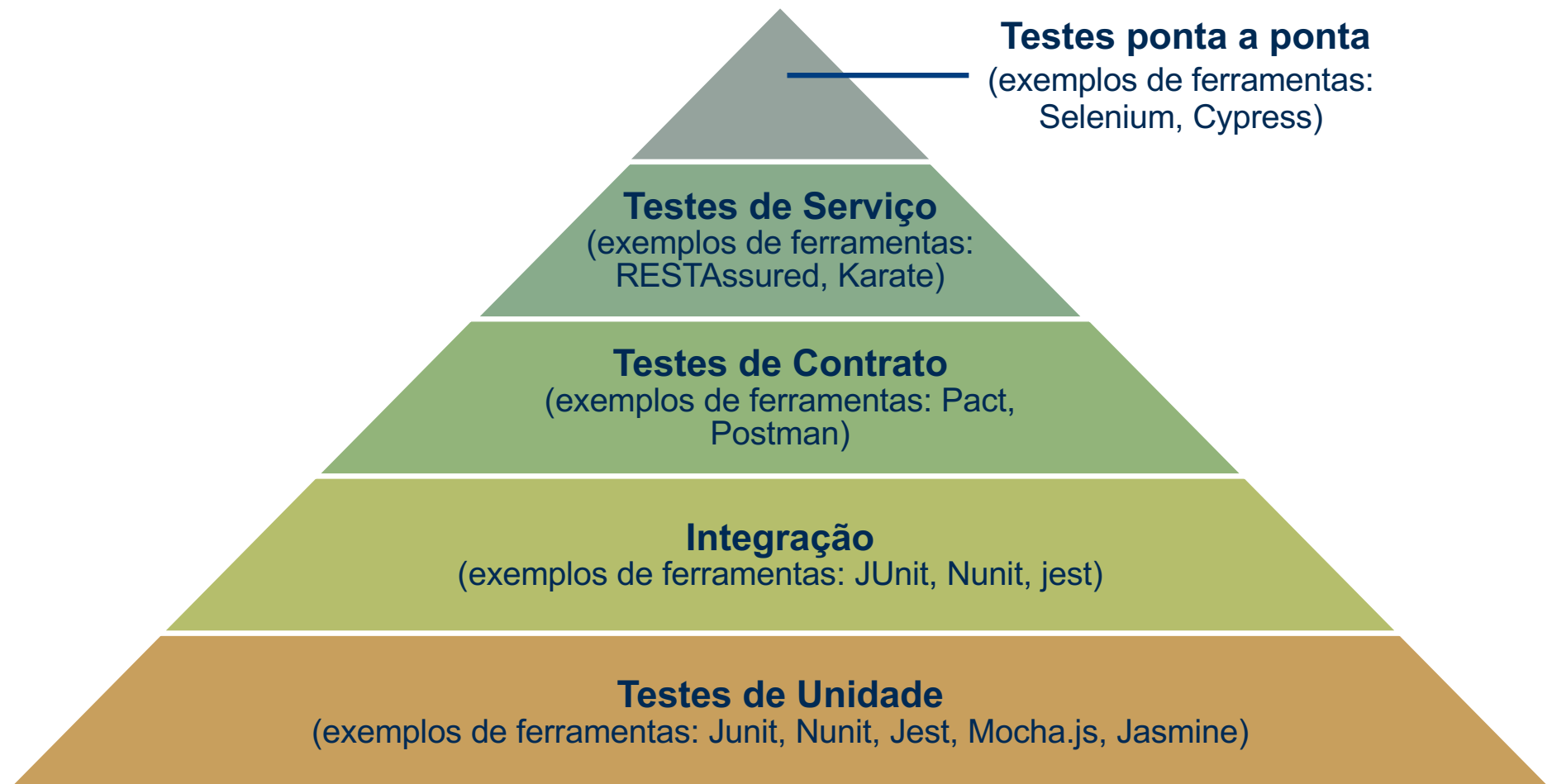
Testes de Software: Testes de Unidade

Elaine Venson

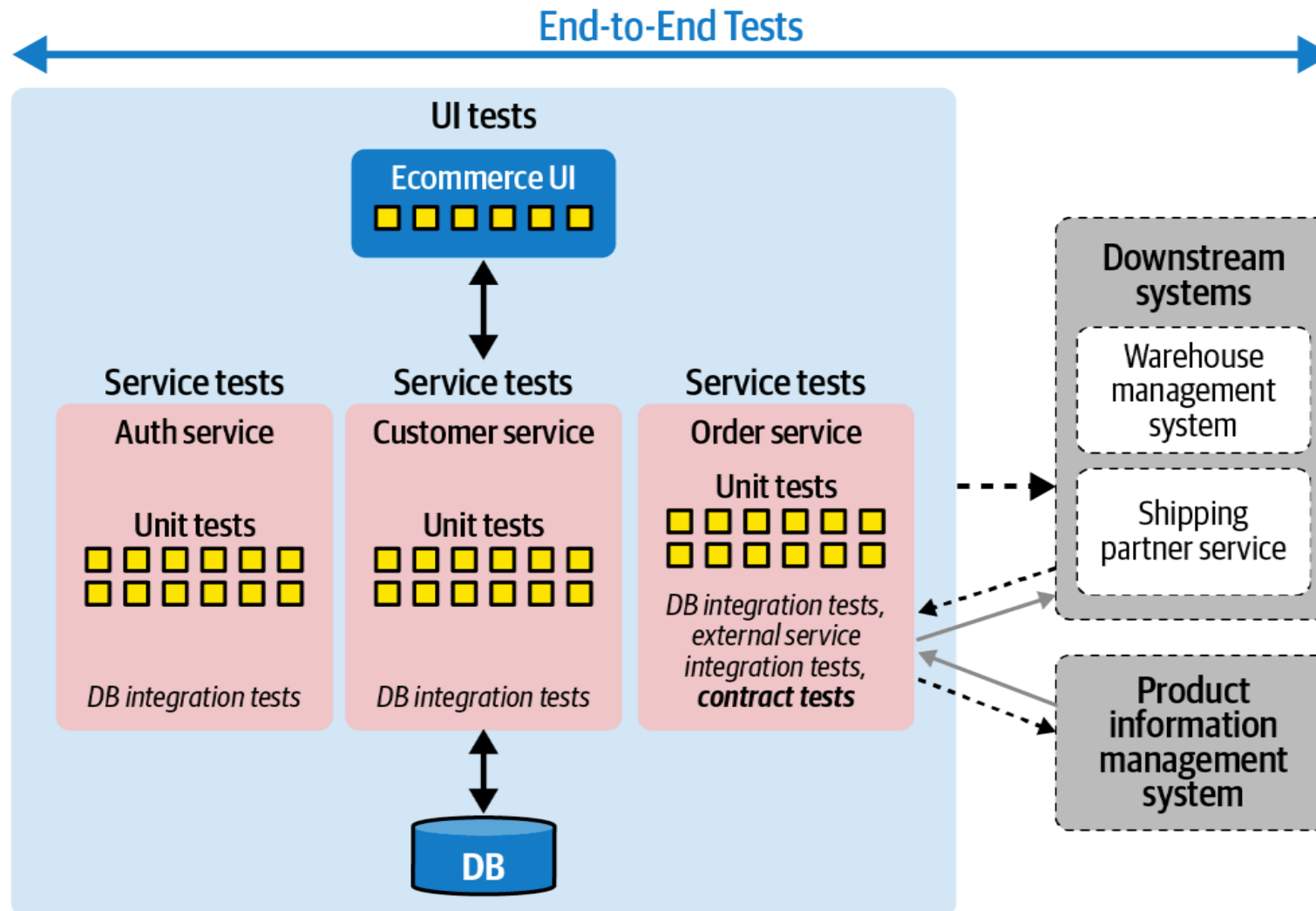
elainevenson@unb.br



Pirâmide de Testes Automatizados: Aplicação Web Orientada a Serviços



Exemplo de uso dos diferentes níveis de teste



Níveis de Teste

- Características dos diferentes níveis:
 - Escopo em que operam
 - Propósito
 - Habilidade em dar feedback
 - Esforço para criar e manter
- Evitam redundância de testes, cada nível trata uma classe específica de defeitos
- Testes nos níveis menores são mais rápidos

Níveis de Teste

- Considerações:
 - Os níveis não são necessariamente sequenciais.
 - As características do sistema a ser testado irão determinar:
 - Níveis de teste
 - Tipos de testes
 - Técnicas de derivação de casos de teste



Testes de Unidade

- Verificam as **menores porções de funcionalidade** de uma aplicação
- Funcionam como uma rede de proteção no nível **micro**
- Têm como objetivo verificar **o menor elemento testável** de um software
- Geralmente realizado pelo **desenvolvedor**
- Casos de teste com foco nos algoritmos e lógica de programação
- Motivações:
 - Gerenciar os elementos combinados do teste, começando pela menor unidade
 - Facilitar o processo de depuração (localização do defeito)
 - Permitir paralelismo da atividade de teste

Exemplo Simples

- Código

```
public class PessoaService {  
    public boolean ehMaiorDeIdade(int idade) {  
        return idade >= 18;  
    }  
}
```

- Testes

```
@Test  
void deveRetornarTruePara18Anos() {  
    assertTrue(service.ehMaiorDeIdade(18));  
}  
  
@Test  
void deveRetornarFalsePara17Anos() {  
    assertFalse(service.ehMaiorDeIdade(17));  
}
```

Testes de Unidade

- Objetivo: comparar a unidade com sua especificação funcional ou de interface, demonstrar que a unidade “não” atende à sua especificação.
- Questões importantes:
 - Como elaborar os casos de teste
 - A ordem em que as unidades devem ser testadas e integradas
 - A execução dos testes

Elaboração dos Casos de Teste

- Informações necessárias:
 - A especificação da unidade (definição das entradas e saídas)
 - Código fonte da unidade
- Procedimentos:
 1. Analisar o código usando métodos caixa-branca.
 2. Complementar com casos de teste caixa-preta baseados na especificação.
- OU
 1. Criar testes a partir das abordagens caixa-preta.
 2. Analisar a cobertura e complementar com testes caixa-branca



O Padrão AAA

Arrange • Act • Assert

- Estrutura simples para organizar testes
- Melhora legibilidade e manutenção
- Facilita entendimento do comportamento testado

Um exemplo

Algum problema com esse teste?

```
@Test
void teste() {
    Calculadora c = new Calculadora();
    int x = c.somar(2,3);
    assertEquals(5, x);
}
```

- Onde estamos preparando o cenário? (Arrange)
- Onde acontece a ação? (Act)
- Onde verificamos o resultado? (Assert)

Arrange

Preparar o cenário do teste:

- criar objetos
- inicializar variáveis
- configurar mocks
- definir entradas

O que preciso preparar para executar o teste?

```
Calculadora calc = new Calculadora();  
int a = 2;  
int b = 3;
```

Act

Executar a ação principal:

- Chamada para a função ou método a ser testado
- Um bom teste normalmente possui apenas UM Act

Qual comportamento estou executando?

```
int resultado = calc.somar(a, b);
```

Assert

Verificar o comportamento esperado

- O assert responde:

O resultado está correto?

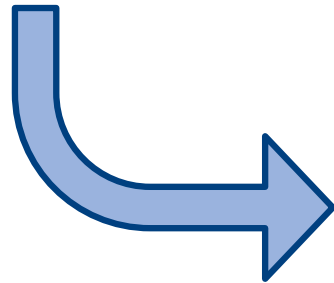
O comportamento esperado aconteceu?

Houve erro/exceção?

```
assertEquals(5, resultado);
```

Com e sem AAA

```
@Test
void teste() {
    Calculadora c = new Calculadora();
    int x = c.somar(2,3);
    assertEquals(5, x);
}
```



```
@Test
void deveSomarDoisNumeros() {

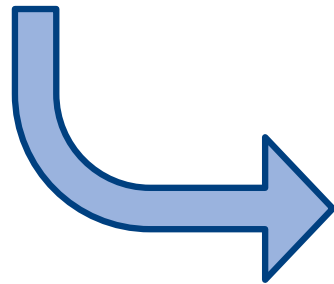
    // Arrange
    Calculadora calc = new Calculadora();

    // Act
    int resultado = calc.somar(2, 3);

    // Assert
    assertEquals(5, resultado);
}
```

Com e sem AAA

```
@Test
void teste1() {
    C c = new C();
    assertEquals(5, c.s(2,3));
}
```



```
@Test
void deveSomarDoisNumeros() {

    // Arrange
    Calculadora calc = new Calculadora();

    // Act
    int resultado = calc.somar(2,3);

    // Assert
    assertEquals(5, resultado);
}
```


AAA Testando Exceções

```
@Test
void deveLancarExcecaoParaValorNegativo() {

    // Arrange
    Conta conta = new Conta();

    // Act + Assert
    assertThrows(
        IllegalArgumentException.class,
        () -> conta.sacar(-10)
    );
}
```

Benefícios de usar AAA

Benefícios

- Maior legibilidade
- Melhor comunicação
- Facilita manutenção
- Reduz confusão
- Facilita revisão de testes
- Ajuda iniciantes

Independência e Isolamento dos Casos de Teste

- **Um teste unitário deve:**
 - funcionar sozinho
 - não depender de outros testes
 - produzir sempre o mesmo resultado
 - testar apenas uma unidade de comportamento

Um teste deve poder executar sozinho, em qualquer ordem!

Evitar dependências externas

Em vez de usar:

- Banco de dados real
- API real
- Arquivos
- Relógio do sistema

Tornam os testes lentos, frágeis e imprevisíveis!

usar:

- Dublês de teste

Evitar dependências externas

Código:

```
public class EmailService {  
  
    private ServidorEmail servidor;  
  
    public EmailService(ServidorEmail servidor) {  
        this.servidor = servidor;  
    }  
  
    public void enviarBoasVindas() {  
        servidor.enviar("Bem-vindo!");  
    }  
}
```

Teste com dublê:

```
@Test  
void deveEnviarMensagemDeBoasVindas() {  
    ServidorEmail servidor = mock(ServidorEmail.class);  
  
    EmailService service = new EmailService(servidor);  
  
    service.enviarBoasVindas();  
  
    verify(servidor).enviar("Bem-vindo!");  
}
```

Nomeando Casos de Teste

O nome do teste deve explicar:

- o comportamento verificado,
- o cenário testado,
- o resultado esperado.

Bons nomes ajudam a:

- entender rapidamente o teste,
- documentar comportamento,
- facilitar manutenção,
- reduzir esforço cognitivo.

Um teste também é documentação

“Os testes explicam como o sistema deve se comportar.”

Nomeando Casos de Teste

Convenção muito usada:

`[Metodo]_[Cenario]_[ResultadoEsperado]`

Exemplo:

`Sum_TwoNumbers_ReturnsSum()`

Problemas:

- foca na implementação
- repete detalhes desnecessários
- dificulta leitura
- não comunica comportamento claramente

Nomeando Casos de Teste

Use frases simples e expressivas:

```
Delivery_with_a_past_date_is_invalid()
```

Objetivo:

Descrever o **comportamento** de forma clara, direta e compreensível.

-> O método pode mudar de nome, o comportamento esperado não.

Boas práticas para nomes de testes

- ✓ Use linguagem simples:

```
User_with_invalid_password_cannot_login()
```

- ✓ Separe palavras com underscore:

```
Cart_with_no_items_has_total_zero()
```

- ✓ Expresse comportamento:

```
Order_above_100_receives_free_shipping()
```

- ✓ Seja específico:

```
Delivery_with_a_past_date_is_invalid()
```

A execução dos testes unitários

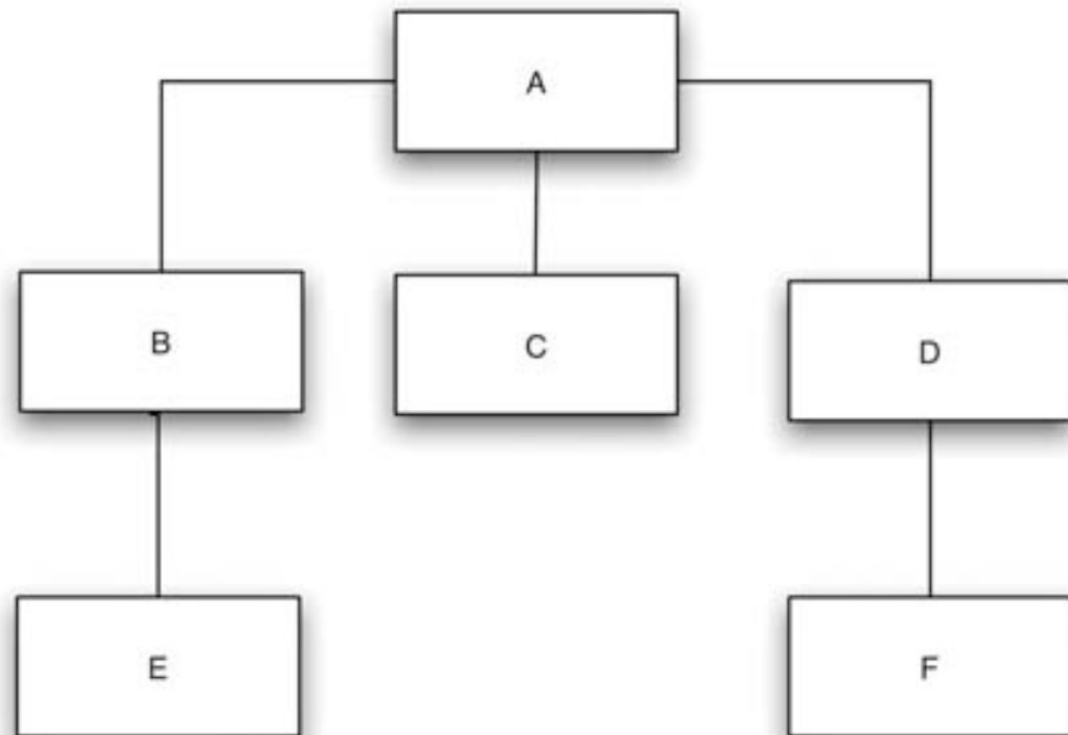
- Dicas e orientações:
 - A definição da saída esperada faz parte do caso de teste. Quando o resultado da execução de um caso de teste não bate com o resultado esperado, há duas possibilidades:
 - A unidade contém um defeito.
 - O resultado esperado do caso de teste está errado.
 - Quando executar um caso de teste, verificar efeitos colaterais não esperados, ex.: valores que não deveriam ter sido alterados.
 - Ferramentas de teste podem ajudar a automatizar tarefas rotineiras e desinteressantes. Ex.: ferramentas que eliminam a necessidade de drivers, ferramentas de análise de fluxo, etc.

A execução dos testes unitários

- Dicas e orientações (cont):
 - Em vez de testar o próprio código, desenvolvedores podem trocar as unidades a serem testadas. O programador que chama um componente é um bom candidato para testar o componente.
 - Evitar casos de teste descartáveis, melhor representá-los de uma forma que possam ser reutilizados.
 - Se muitos defeitos foram encontrados em uma unidade, é provável que ela possua ainda mais defeitos não detectados. Tais unidades precisam passar por testes adicionais.
 - Lembrar que o objetivo do teste não é demonstrar que ele funciona corretamente, mas sim demonstrar a presença de defeitos.

A ordem dos testes

- **Teste Não-Incremental:**
cada unidade é testada independentemente
- **Teste Incremental:**
combinar o próximo teste com uma unidade previamente testada
 - Bottom-up
 - Top-down



A ordem dos testes de unidade

NÃO INCREMENTAL

- Requer mais esforço (criação de drivers e stubs)
- Menor tempo de execução
- Permite mais testes simultâneos

INCREMENTAL

- Menor esforço (uso de unidades testadas)
- Detecta defeitos de integração mais cedo
- Debugging mais fácil
- Código é mais exercitado

Atividade 1 - Anatomia dos testes do projeto

Objetivo

- Analisar como o projeto open-source organiza seus testes

Tarefa

Cada grupo deve identificar:

- Convenção de nomes
- Estrutura AAA
- Uso de mocks/stubs
- Dependências externas

Atividade 2 - Encontrar um “teste ruim”

Tarefa

Escolher um teste do projeto que:

- seja difícil de entender,
- tenha múltiplos asserts,
- misture Arrange/Act/Assert,
- dependa de infraestrutura,
- tenha nomes ruins.



Atividade 3 – Reescreva um teste

Tarefa

Reescrever um teste ruim do projeto:

- Usando AAA,
- Cobrindo valores limite,
- Melhorando nomenclatura

Entrega

1. Organização dos testes no projeto
 - Convenção de nomes
 - Estrutura AAA
 - Uso de mocks/stubs
 - Dependências externas
2. Testes ruins, com descrição dos problemas
3. Testes reescritos, com descrição das melhorias realizadas

Referências

- Myers, G. J. et al, The Art of Software Testing. Ed. John Wiley & Sons, 2012.
(Capítulo 5)